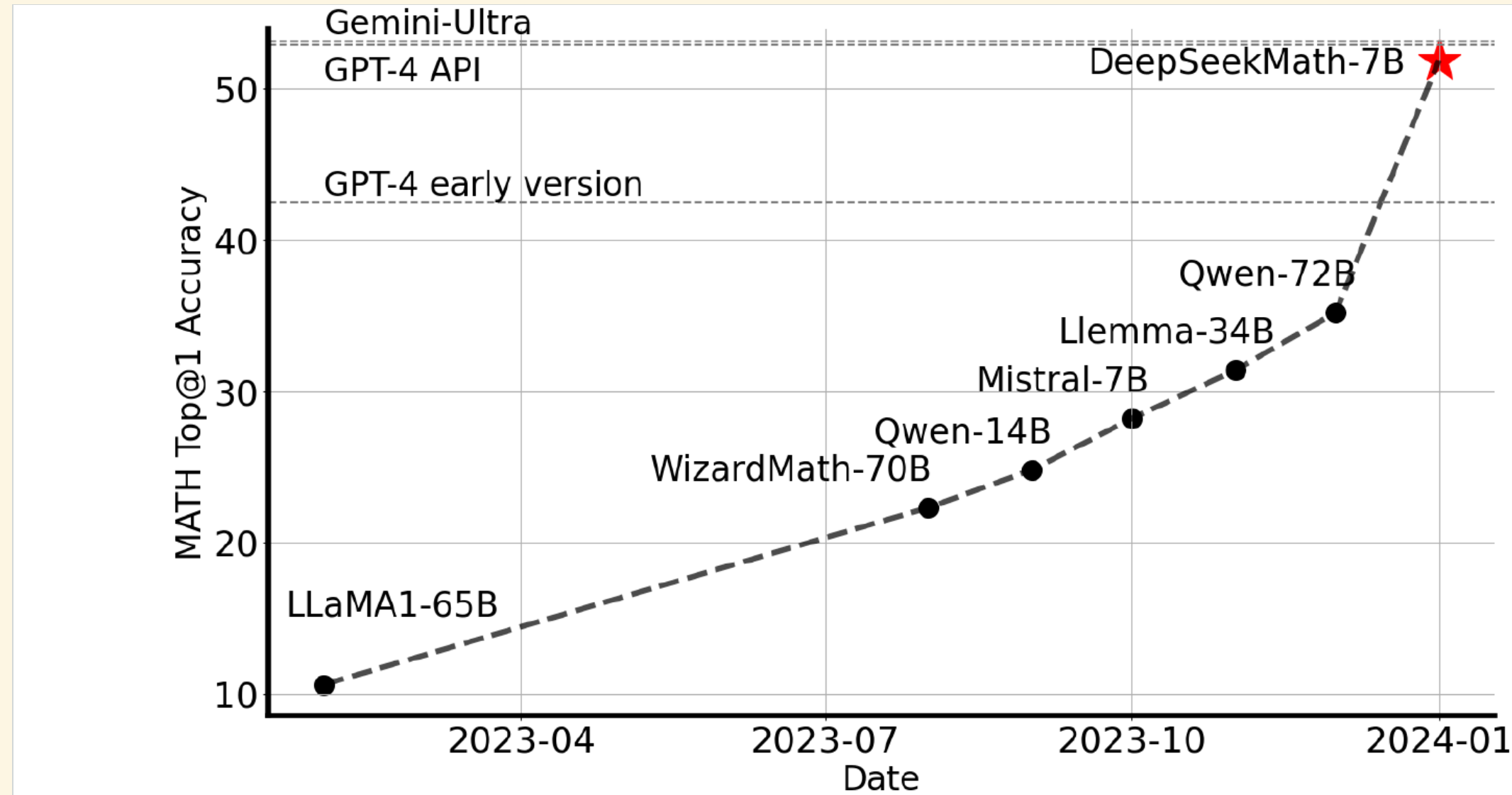


DeepSeekMath

Pushing the Limits of Mathematical Reasoning in Open Language Models

Dhruv Kumar · BITS Pilani

A 7B open model approaching GPT-4 on competition math



Shao et al. (2024) train **DeepSeekMath-RL 7B**, which reaches **51.7%** Top-1 accuracy on the competition-level MATH benchmark — no tools, no voting — approaching GPT-4 and Gemini-Ultra, and far ahead of every other open 7B–70B model. Two contributions explain this: a meticulously engineered **pretraining corpus**, and **GRPO** — the RL algorithm this paper introduces, later used to train DeepSeek-R1.

Two contributions, in the order they're built

1. DeepSeekMath Corpus

120B math tokens mined from Common Crawl via an iterative classifier pipeline — 2.3× larger than any prior public math pretraining corpus.

2. GRPO

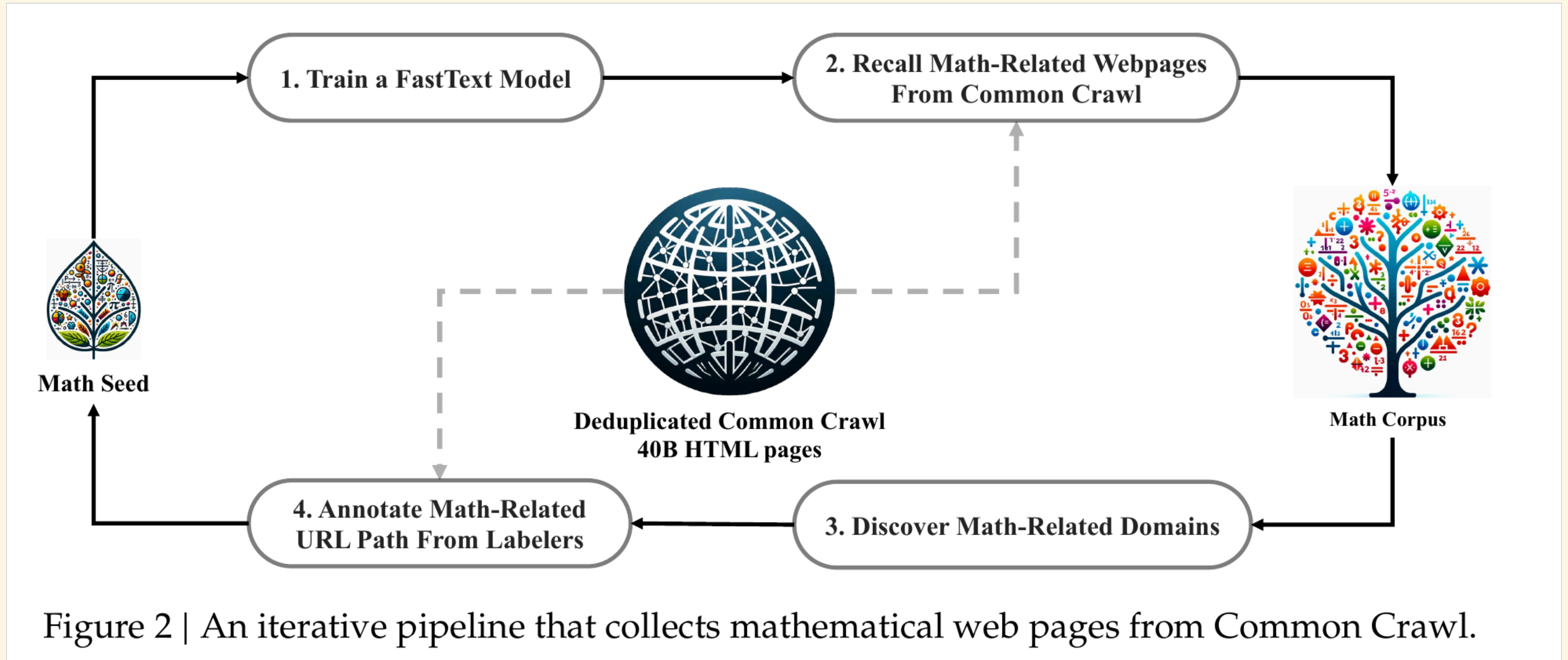
Group Relative Policy Optimization — a lighter-weight alternative to PPO that drops the value model entirely, introduced here and used unchanged in DeepSeek-R1.

A unifying lens

The paper also reframes SFT, RFT, DPO, PPO, and GRPO as one family of algorithms differing in just three choices — useful for the whole rest of this deck.

DeepSeekMath-Base 7B starts from **DeepSeek-Coder-Base-v1.5**, not a general LLM — a deliberate choice this deck returns to later.

Mining 120B math tokens: the iterative pipeline



Four stages, repeated in a loop: **(1)** train a fastText classifier on a seed corpus, **(2)** use it to recall math-like pages from Common Crawl, **(3)** find whole domains where recall was high (e.g. mathoverflow.net), **(4)** have human annotators mark the specific math-related URL paths within those domains – which then enlarge the seed corpus for the next round.

Intuition: bootstrapping a classifier with itself

A classifier trained once on a small, narrow seed set (OpenWebMath) only recognizes pages that *look like* that seed set — it misses math content in a different style or domain entirely. The fix is to let the classifier's own blind spots point to where to look next.

WHY DOMAINS, NOT INDIVIDUAL PAGES

Checking every one of 40B pages by hand is impossible. But if >10% of a *domain's* pages were already recalled, that whole domain is very likely math-related — annotators only need to mark the URL *patterns* worth keeping (e.g. `mathoverflow.net/questions`), and every matching page is added at once. This converts millions of individual judgments into a handful of domain-level ones.

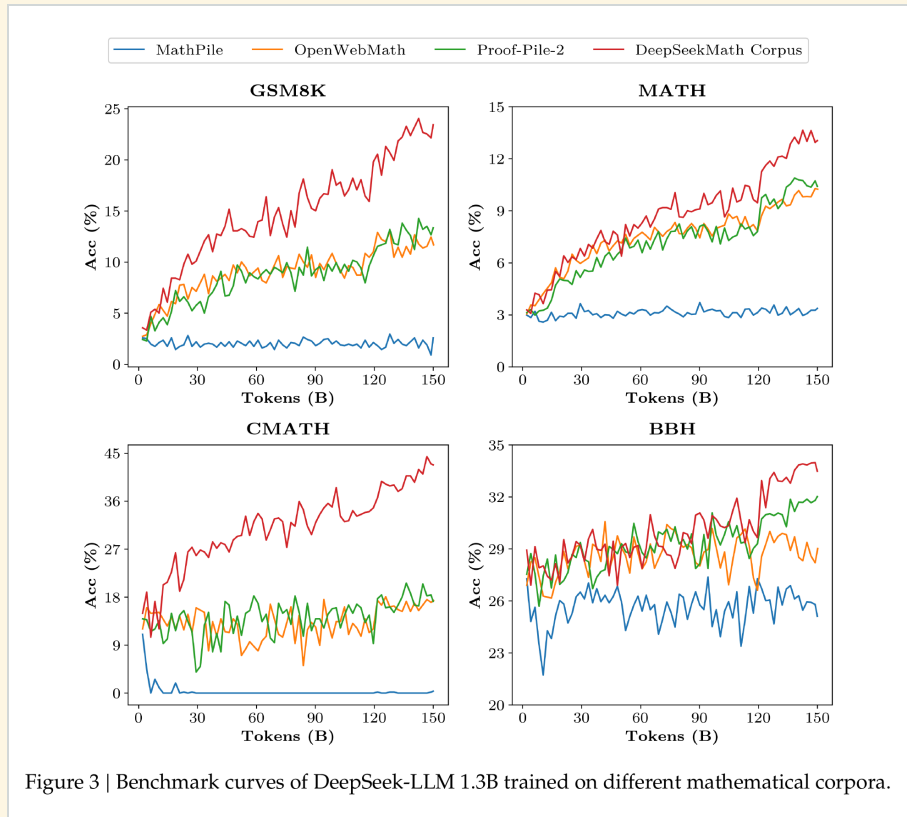
After 4 iterations: **35.5M web pages, 120B tokens** — nearly **9×** the size of OpenWebMath (120.2B vs. 13.6B tokens, verified). Iteration 4 was stopped because 98% of its pages were already found in iteration 3 — the seed corpus had converged.

Does the extra scale actually help? Yes.

Math Corpus	Size	English Benchmarks					Chinese Benchmarks		
		GSM8K	MATH	OCW	SAT	MMLU STEM	CMATH	Gaokao MathCloze	Gaokao MathQA
No Math Training	N/A	2.9%	3.0%	2.9%	15.6%	19.5%	12.3%	0.8%	17.9%
MathPile	8.9B	2.7%	3.3%	2.2%	12.5%	15.7%	1.2%	0.0%	2.8%
OpenWebMath	13.6B	11.5%	8.9%	3.7%	31.3%	29.6%	16.8%	0.0%	14.2%
Proof-Pile-2	51.9B	14.3%	11.2%	3.7%	43.8%	29.2%	19.9%	5.1%	11.7%
DeepSeekMath Corpus	120.2B	23.8%	13.6%	4.8%	56.3%	33.1%	41.5%	5.9%	23.6%

Same 1.3B model, same 150B training tokens, only the corpus changes. The DeepSeekMath Corpus wins on every single benchmark — including the Chinese ones (CMATH, Gaokao), where English-only corpora like Proof-Pile-2 barely help at all, since this corpus was deliberately built to be multilingual.

Bigger doesn't just mean "same curve, further" — it's a better curve



MathPile and OpenWebMath are so much smaller that 150B tokens means repeating them many times — performance plateaus fast. The DeepSeekMath Corpus is large enough that the model is still seeing mostly *new* text late into training, so the curve keeps climbing instead of flattening.

DeepSeekMath-Base 7B vs. a 77×-larger closed model

Model	Size	English Benchmarks					Chinese Benchmarks		
		GSM8K	MATH	OCW	SAT	MMLU STEM	CMATH	Gaokao MathCloze	Gaokao MathQA
Closed-Source Base Model									
Minerva	7B	16.2%	14.1%	7.7%	-	35.6%	-	-	-
Minerva	62B	52.4%	27.6%	12.0%	-	53.9%	-	-	-
Minerva	540B	58.8%	33.6%	17.6%	-	63.9%	-	-	-
Open-Source Base Model									
Mistral	7B	40.3%	14.3%	9.2%	71.9%	51.1%	44.9%	5.1%	23.4%
Llemma	7B	37.4%	18.1%	6.3%	59.4%	43.1%	43.4%	11.9%	23.6%
Llemma	34B	54.0%	25.3%	10.3%	71.9%	52.9%	56.1%	11.9%	26.2%
DeepSeekMath-Base	7B	64.2%	36.2%	15.4%	84.4%	56.5%	71.7%	20.3%	35.3%

DeepSeekMath-Base 7B beats every open baseline on all 8 benchmarks, and even outperforms **Minerva 540B** — a closed model 77× larger, built on PaLM and further trained on math text. Parameter count isn't the only lever; corpus quality is another.

Why start from a code model? A controlled test

Training Setting	Training Tokens			w/o Tool Use			w/ Tool Use	
	General	Code	Math	GSM8K	MATH	CMATH	GSM8K+Python	MATH+Python
No Continual Training	–	–	–	2.9%	3.0%	12.3%	2.7%	2.3%
Two-Stage Training								
Stage 1: General Training	400B	–	–	2.9%	3.2%	14.8%	3.3%	2.3%
Stage 2: Math Training	–	–	150B	19.1%	14.4%	37.2%	14.3%	6.7%
Stage 1: Code Training	–	400B	–	5.9%	3.6%	19.9%	12.4%	10.0%
Stage 2: Math Training	–	–	150B	21.9%	15.3%	39.7%	17.4%	9.4%
One-Stage Training								
Math Training	–	–	150B	20.5%	13.1%	37.6%	11.4%	6.5%
Code & Math Mixed Training	–	400B	150B	17.6%	12.1%	36.3%	19.7%	13.5%

Same recipe, two different first stages: 400B tokens of **code** vs. 400B tokens of generic **text**, both followed by 150B tokens of math. Code-first wins on every column — GSM8K 21.9% vs. 19.1%, GSM8K+Python 17.4% vs. 14.3% — a controlled answer to "does code training help math reasoning," at least in this domain.

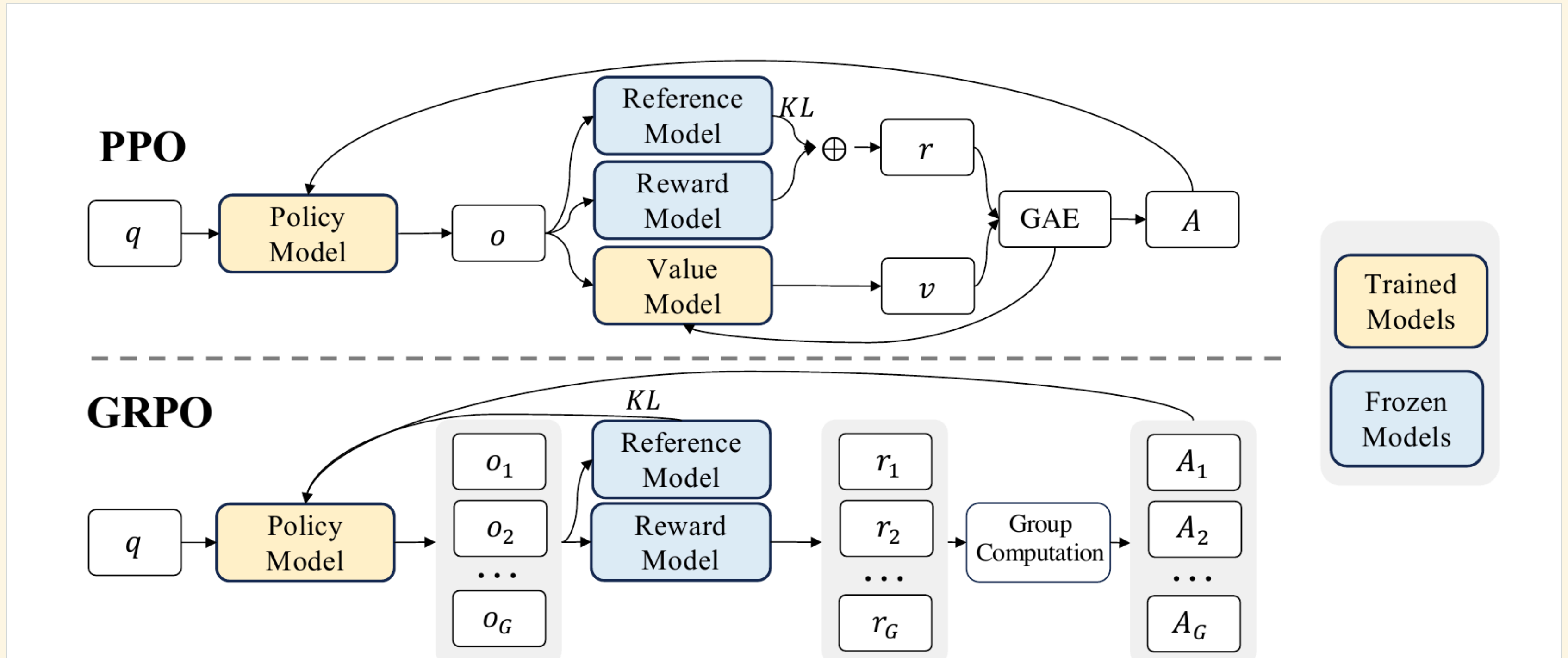
A negative result worth keeping: arXiv papers don't help

ArXiv papers are a standard ingredient in prior math corpora (Proof-Pile-2, Minerva, MathPile) on the assumption that formal mathematical writing teaches formal mathematical reasoning. Training DeepSeek-LLM 1.3B and DeepSeek-Coder-Base-v1.5 7B on **arXiv-only** corpora (MathPile, ArXiv-RedPajama) shows **no improvement, and sometimes deterioration**, across GSM8K, MATH, MMLU-STEM, and formal theorem proving (miniF2F).

WHY THIS IS WORTH REPORTING, NOT JUST DISCARDING

The authors explicitly flag this as preliminary — untested at larger scale, and untested combined with other data types. A clearly-scoped negative result is still useful: it means "arXiv papers = better math reasoning" should not be assumed without evidence.

From PPO to GRPO: what gets removed



In PPO, the **value model** — another network roughly the size of the policy — estimates a baseline reward for variance reduction, and needs its own training. GRPO replaces it entirely with something free: sample several responses to the *same* prompt, and use their own **group average** as the baseline. No extra model, no extra training.

Intuition: why a group average is a good baseline

PPO's value model has to answer a hard, general question: "starting from *this exact partial response*, how good an outcome should I expect?" — for every token, for every prompt, requiring a trained predictor.

GRPO SIDESTEPS THE QUESTION ENTIRELY

Instead of predicting what a good outcome looks like in the abstract, GRPO just *generates several actual outcomes* for the same prompt and compares them to each other. The value model's job — "what's a typical reward here?" — is answered empirically by the group itself, at zero extra training cost. This is only cheap because generating a few extra rollouts is far less expensive than training and running a second full-size model at every step.

The paper notes a second benefit: reward models are usually trained on *comparisons* between outputs to the same question in the first place — so grading rollouts relative to their own group matches how the reward signal was built.

The GRPO objective

$$\mathcal{J}_{\text{GRPO}}(\theta) = \mathbb{E} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \min \left(\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t}|q, o_{i,<t})} \hat{A}_{i,t}, \text{clip}(\cdot, 1-\varepsilon, 1+\varepsilon) \hat{A}_{i,t} \right) - \beta D_{KL}(\pi_{\theta} \parallel \pi_{ref}) \right]$$

G — group size, the number of sampled responses per question q ; o_i — the i -th response, with $|o_i|$ tokens; $\pi_{\theta}, \pi_{\theta_{\text{old}}}$ — current and pre-update policy; $\hat{A}_{i,t}$ — group-relative advantage (next slide); ε — clip range; β — KL penalty strength against a fixed reference policy π_{ref} .

This is **token-level** clipping — unlike DeepSeek-R1's later simplification to one ratio per whole response — and the KL term is added directly to the loss, not folded into the reward as PPO does.

Group-relative advantage, and an unbiased KL estimator

$$\hat{A}_{i,t} = \tilde{r}_i = \frac{r_i - \text{mean}(\mathbf{r})}{\text{std}(\mathbf{r})}, \quad D_{KL}(\pi_\theta \parallel \pi_{ref}) = \frac{\pi_{ref}(o_{i,t} \mid q, o_{i,<t})}{\pi_\theta(o_{i,t} \mid q, o_{i,<t})} - \log \frac{\pi_{ref}(o_{i,t} \mid q, o_{i,<t})}{\pi_\theta(o_{i,t} \mid q, o_{i,<t})} - 1$$

$\mathbf{r} = \{r_1, \dots, r_G\}$ — rewards for the whole group; every token in response o_i gets the same advantage $\hat{A}_{i,t}$ under **outcome supervision** (one reward per full response). The KL formula is an unbiased estimator (Schulman, 2020) guaranteed to be non-negative — unlike a naive log-ratio, it can't accidentally reward the policy for drifting from the reference.

A unifying lens: one gradient formula for five algorithms

$$\nabla_{\theta} \mathcal{J}_{\mathcal{A}}(\theta) = \mathbb{E}[\underbrace{(q, o) \sim \mathcal{D}}_{\text{Data Source}}] \left(\frac{1}{|o|} \sum_{t=1}^{|o|} \underbrace{GC_{\mathcal{A}}(q, o, t, \pi_{rf})}_{\text{Gradient Coefficient}} \nabla_{\theta} \log \pi_{\theta}(o_t | q, o_{<t}) \right). \quad (5)$$

SFT, RFT, DPO, PPO, and GRPO all turn out to be the *same equation* — the only differences are: where the training data (q, o) comes from (the **data source**), and how strongly each token gets pushed up or down (the **gradient coefficient** GC). Everything from here is just filling in these two blanks differently.

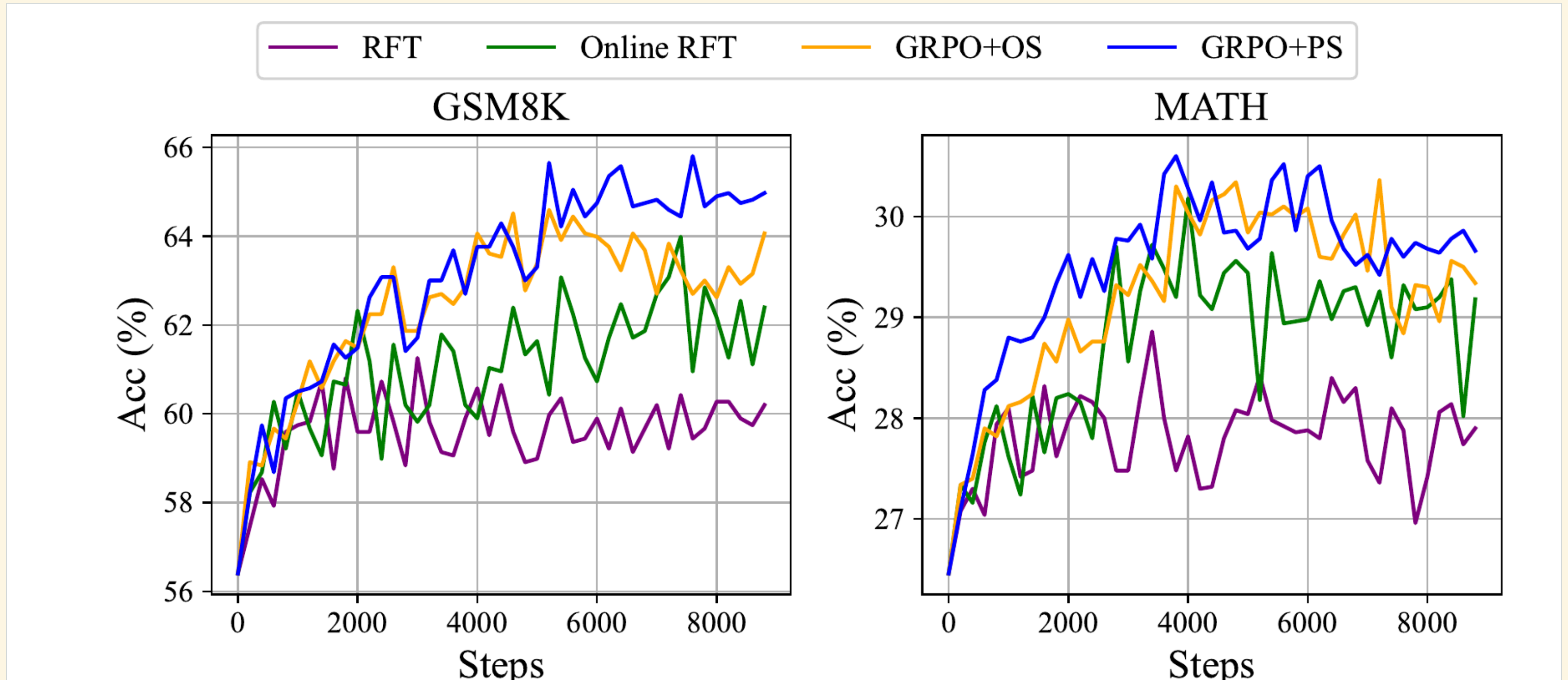
Filling in the blanks, method by method

Methods	Data Source	Reward Function	Gradient Coefficient
SFT	$q, o \sim P_{sft}(Q, O)$	-	1
RFT	$q \sim P_{sft}(Q), o \sim \pi_{sft}(O q)$	Rule	Equation 10
DPO	$q \sim P_{sft}(Q), o^+, o^- \sim \pi_{sft}(O q)$	Rule	Equation 14
Online RFT	$q \sim P_{sft}(Q), o \sim \pi_{\theta}(O q)$	Rule	Equation 10
PPO	$q \sim P_{sft}(Q), o \sim \pi_{\theta}(O q)$	Model	Equation 18
GRPO	$q \sim P_{sft}(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta}(O q)$	Model	Equation 21

SFT has a fixed coefficient of 1 — every human-written token is pushed equally hard, unconditionally. **RFT/Online RFT** use a binary indicator — 1 if the sampled answer was correct, 0 otherwise; no notion of "how much better." **GRPO** is the only one whose coefficient is a continuous, signed number that can differentially reward and *penalize* responses by degree — that extra expressiveness is exactly what the next two slides show mattering in practice.

The data-source axis: online beats offline

Offline methods (RFT, DPO) train on responses sampled once from the fixed, frozen SFT model. **Online** methods (Online RFT, GRPO) resample from the *current*, continuously-updating policy at every step.



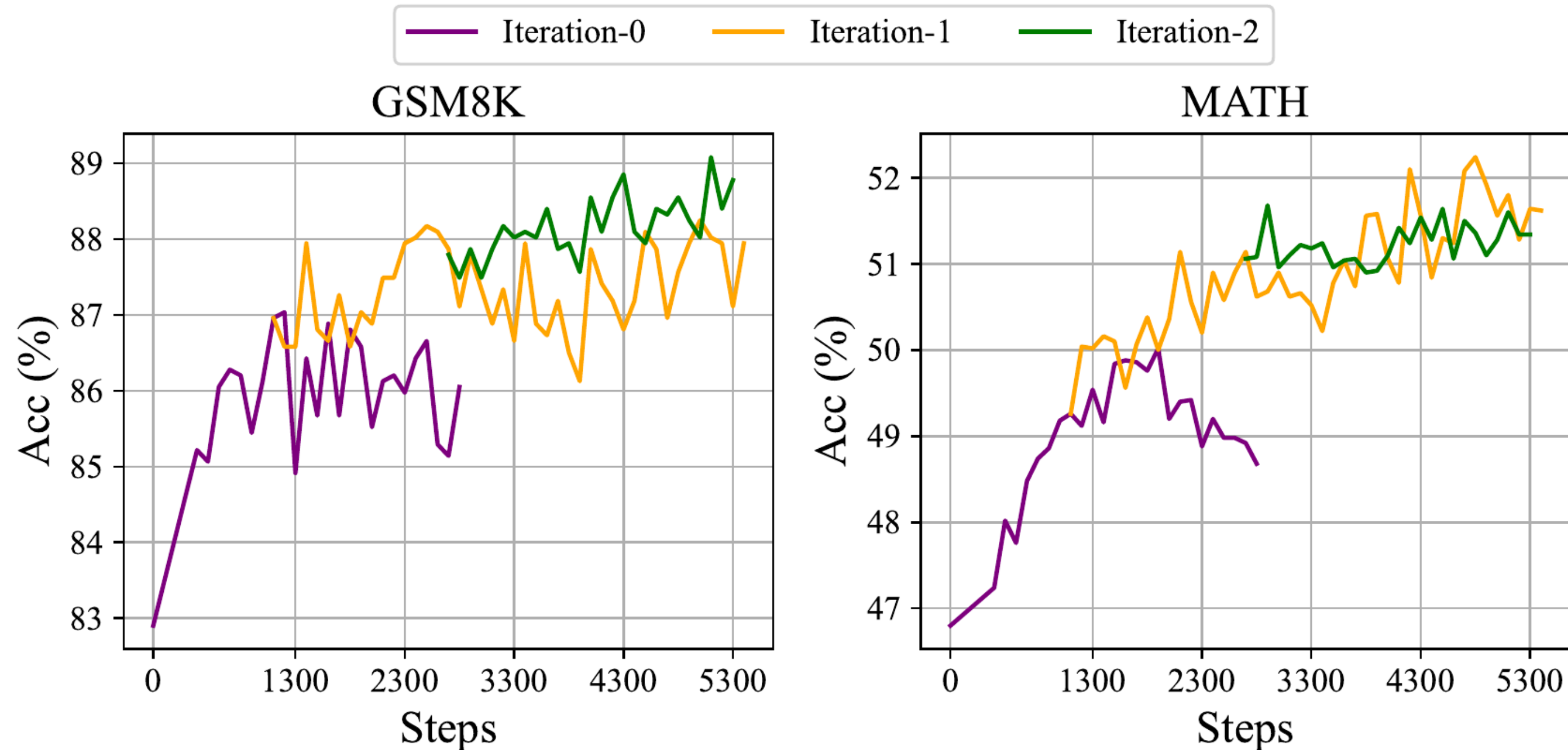
Early in training the SFT model and the current policy are nearly identical, so online sampling barely matters yet. As training progresses they diverge, and online methods pull ahead — Online RFT clearly beats offline RFT on both benchmarks.

The gradient-coefficient axis: GRPO beats Online RFT

Online RFT and GRPO share the same online data source — the only difference is the gradient coefficient. Online RFT's binary indicator reinforces every correct rollout **equally**, and does nothing for incorrect ones. GRPO's continuous advantage reinforces *more strongly* for above-average rollouts and actively **penalizes** below-average ones.

Figure 5 (previous slide) shows GRPO clearly ahead of Online RFT on both GSM8K and MATH — this gap is the direct payoff of the richer gradient coefficient. GRPO+PS (process supervision — one reward per reasoning *step*, not just the final answer) edges out GRPO+OS further, showing fine-grained credit assignment helps too.

Iterative RL: retraining the reward model mid-flight



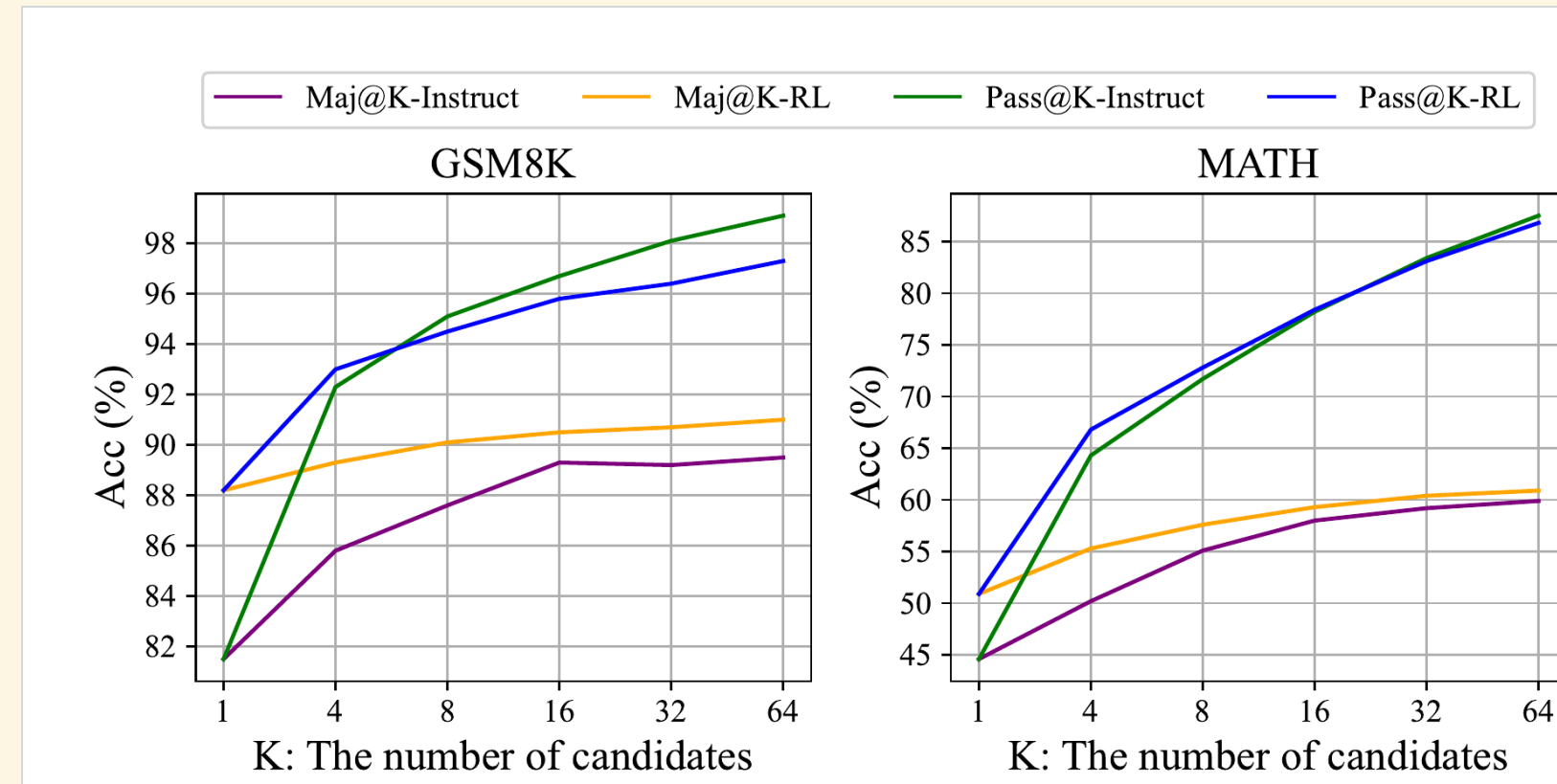
As the policy improves, the old reward model — trained on an earlier, weaker policy's outputs — starts to see rollouts unlike anything in its own training data, and its judgments grow less trustworthy. **Iterative GRPO** periodically retrains the reward model on fresh rollouts (keeping 10% older data via replay) and resets the reference policy, and training resumes. Iteration-1 shows the largest jump — the first update is where the reward model was staling fastest.

DeepSeekMath-RL: verified end-to-end gains

Model	Size	English Benchmarks		Chinese Benchmarks	
		GSM8K	MATH	MGSM-zh	CMATH
Chain-of-Thought Reasoning					
Closed-Source Model					
Gemini Ultra	-	94.4%	53.2%	-	-
GPT-4	-	92.0%	52.9%	-	86.0%
Inflection-2	-	81.4%	34.8%	-	-
GPT-3.5	-	80.8%	34.1%	-	73.8%
Gemini Pro	-	86.5%	32.6%	-	-
Grok-1	-	62.9%	23.9%	-	-
Baichuan-3	-	88.2%	49.2%	-	-
GLM-4	-	87.6%	47.9%	-	-
Open-Source Model					
InternLM2-Math	20B	82.6%	37.7%	-	-
Qwen	72B	78.9%	35.2%	-	-
Math-Shepherd-Mistral	7B	84.1%	33.0%	-	-
WizardMath-v1.1	7B	83.2%	33.0%	-	-
DeepSeek-LLM-Chat	67B	84.1%	32.6%	74.0%	80.3%
MetaMath	70B	82.3%	26.6%	66.4%	70.9%
SeaLLM-v2	7B	78.2%	27.5%	64.8%	-
ChatGLM3	6B	72.3%	25.7%	-	-
WizardMath-v1.0	70B	81.6%	22.7%	64.8%	65.4%
DeepSeekMath-Instruct	7B	82.9%	46.8%	73.2%	84.6%
DeepSeekMath-RL	7B	88.2%	51.7%	79.6%	88.8%
Tool-Integrated Reasoning					

GRPO is applied to DeepSeekMath-Instruct using *only* the chain-of-thought instruction-tuning data for GSM8K and MATH — nothing new. Verified gains: GSM8K **82.9→88.2** (+5.3), MATH **46.8→51.7** (+4.9), CMATH **84.6→88.8** (+4.2) — CMATH is *out-of-domain* for the RL data, showing the improvement generalizes beyond exactly what was trained on.

Why does RL work? Maj@K improves, Pass@K doesn't



Pass@K = is a correct answer *somewhere* among K samples? **Maj@K** = does the most *common* answer among K samples happen to be correct? RL barely moves Pass@K — the model's underlying ability to *ever* produce a correct answer is basically unchanged. It substantially improves Maj@K.

THE PAPER'S OWN INTERPRETATION

RL isn't teaching new reasoning capability here — it's making the correct answer more *likely to dominate* the output distribution, so that simple top-of-distribution decoding surfaces it more often. A sharpening of what the model already knew, not necessarily new knowledge.

Open threads, in the unified paradigm's own terms

The paper closes by mapping its own limitations onto the three components of Equation 5:

Data source

Only in-distribution instruction-tuning questions were used, with naive sampling — likely why Pass@K didn't move. Out-of-distribution prompts and better decoding (tree search) are flagged as next steps.

Algorithm

Every method here fully trusts its reward signal — but even carefully human-annotated process-reward data (PRM800K) has ~20% labeling errors. Algorithms robust to noisy rewards are an open problem.

Reward function

Generalizing reward models to out-of-distribution outputs, quantifying their uncertainty, and building better process-level (step-by-step) reward models.

Reading list

- [Shao, Wang, Zhu, Xu, Song, Bi, Zhang, Zhang, Li, Wu & Guo, *DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models*](#) — the source for every figure and result in this deck.
- [Schulman, Wolski, Dhariwal, Radford & Klimov, *Proximal Policy Optimization Algorithms*](#) — the PPO algorithm GRPO simplifies.
- [Raschka, *The State of Reinforcement Learning for LLM Reasoning*](#) — a detailed side-by-side of PPO vs. GRPO.
- [Yamani, *Advanced Understanding of Group Relative Policy Optimization \(GRPO\) in DeepSeekMath*](#) — Hugging Face LLM Course, worked intuition for the advantage, clipping, and KL terms.
- [Sulahian, *Demystifying DeepSeekMath's Data Pipeline*](#) — a reproduction of the fastText iterative mining pipeline.
- [Lambert, *Recent reasoning research: GRPO tweaks, base model RL, and data curation*](#) — Interconnects, surveys GRPO variants that followed this paper.
- [Official DeepSeekMath code, data, and model checkpoints.](#)